Course Code	Course Title					Course Type
CS2201	Design and Analysis of Algorithms					PCC
Prerequisite	Contact Hours per Week			Internal	External	Credits
	L	T	P			
	3	0	0	40	60	3

Course Objectives

- Introduces the notations for analysis of the performance of algorithms.
- Describes major algorithmic techniques (divide-and-conquer, backtracking, dynamic programming, greedy, branch and bound methods) and mention problems for which each technique is appropriate;
- Describes how to evaluate and compare different algorithms using worst-, average-, and best-case analysis.
- Explains the difference between tractable and intractable problems, and introduces the problems that are P, NP and NPcomplete.

Course Outcomes

1. Students will understand the basic concepts of algorithm design and analysis.
2. Students will be able to analyze and evaluate algorithm performance using asymptotic notations.
3. Students will learn to apply various algorithmic techniques such as divide and conquer, dynamic programming, and greedy method.
4. Students will gain proficiency in solving problems using backtracking and branch and bound methods.
5. Students will gain an understanding of NP-hard and NP-complete problems and their implications in the real world.

Detailed Contents

UNIT-I

Introduction:

Notion of an Algorithm – Fundamentals of Algorithmic Problem Solving – Important Problem Types – Fundamentals of the Analysis of Algorithmic Efficiency –Asymptotic Notations and their properties. Analysis Framework – Empirical analysis – Mathematical analysis for Recursive and Non-recursive algorithms – Visualization

UNIT-II

Brute Force – String Matching , Closest-Pair and Convex-Hull Problems.
Exhaustive Search – Traveling Salesman Problem, Knapsack Problem , Assignment problem.
Divide and conquer: General method, applications-Binary search, Quick sort, Merge sort, Strassen's

matrix multiplication,convex hull,closest pair, large integer multiplication.

UNIT-III

Dynamic programming – Principle of optimality,Chain Matrix Multiplication, Computing a Binomial Coefficient , Floyd‘s algorithm , Multistage graph , Optimal Binary Search Trees , Knapsack Problem and Memory functions.

Greedy Technique –Prim‘s algorithm and Kruskal’s Algorithm , Fractional Knapsack problem, Optimal Merge pattern – Huffman Trees.

UNIT-IV

Backtracking: General method, applications−n-queen problem, sum of subsets problem,graph coloring, Hamiltonian cycles.

Branch and Bound: General method, applications - Traveling Salesperson Problem, 0/1 knapsack problem, Assignment problem - LC Branch and Bound solution, FIFO Branch and Bound solution.

UNIT-V

NP-Hard and NP-Complete problems: NP Hard and NP completeness: Basic concepts, cook’s theorem, NP-hard graph problems and scheduling problem, NP- hard code generation problems, Clique Decision problem, Node covering problem, scheduling problem, NP hard code generation problem.

Approximation Algorithms for NP-Hard Problems – Traveling Salesman problem,Knapsack problem.

Text Books

- Horowitz, E., Sahni, S., and Rajasekaran, S. (2019). Fundamentals of Computer Algorithms. University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2001). Introduction to Algorithms, Second Edition. Pearson Education.

References

1. A. V. Levitin, "Introduction to the Design and Analysis of Algorithms," Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.
2. A. Aho, J. Ullman, and M. Hopcroft, "Design and Analysis of Algorithms," Pearson Education.
3. M. T. Goodrich and R. Tamassia, "Algorithm Design: Foundations, Analysis and Internet Examples," John Wiley and Sons.

Course Code	Course Title					Course Type
CS2801	Design and Analysis of Algorithms Lab					PCC
Prerequisite	Contact Hours per Week			Internal	External	Credits
	L	T	P			
	0	0	3	40	60	1.5

Course Objectives

- To develop proficiency in programming algorithms using C/C++.
- To introduce the fundamental concepts of data structures and algorithms.
- To provide practical experience in implementing various data structures and algorithms.
- To understand the techniques used in solving various computational problems.
- To develop problem-solving skills and logical reasoning.

Course Outcomes

At the end of the course the students will have the

1. Ability to write C/C++ programs for various algorithms and data structures.
2. Knowledge of fundamental concepts and terminology related to data structures and algorithms.
3. Proficiency in implementing basic data structures such as linked lists, trees, and graphs.
4. Ability to analyze the time and space complexity of algorithms using asymptotic notations.
5. Problem-solving skills using different algorithmic paradigms such as divide-and-conquer, dynamic programming, greedy algorithms, and backtracking.

List of Experiments:

All the problems have to be implemented either writing C programs or writing C++ programs. Elementary Problems:

1) Using a stack of characters, convert an infix string to a postfix string.
2) Implement polynomial addition using a single linked list
3) Implement insertion, deletion, searching of a BST, Also write a routine to draw the BST horizontally.
4) Implement binary search and linear search in a program
5) Implement heap sort using a max heap.
6) Implement DFS/ BFS routine in a connected graph
7) Implement Dijkstra's shortest path algorithm using BFS
8) Greedy Algorithm (Any Two)
    a) Given a set of weights, form a Huffman tree from the weight and also find oot the code corresponding to each weight.
    b) Take a weighted graph as an input, find out one MST using Kruskal/ prim's algorithm
    c) Given a set of weight and an upper bound M – Find out a solution to the Knapsack problem
9) Divide and Conquer Algorithm (any Two)
    a) Write a quick sort routine, run it for a different input sizes and calculate the time of running. Plot in graph paper input size versus time.

b) Implement two way merge sort and calculate the time of sorting

c) Implement Strassen's matrix multiplication algorithm for matrices whose order is a power of two.

10) Dynamic programming

a. Given two sequences of character, find out their longest common subsequence using dynamic programming